

CSc 453: Fall 2025

Assignment 2 milestone 3

Start: Oct 3, 2025

Due: 11:59 PM, Oct 9, 2025

General

This assignment involves constructing abstract syntax trees (ASTs) for [G2](#) programs. The next assignment will use these ASTs as the basis for code generation.

Required functionality

1. Programming requirements

Your program should extend the functionality of your parser from Assignment 2 milestone 2 to additionally construct ASTs for programs in the [G2 subset of C--](#):

- Functionality from earlier assignments, i.e., syntax and semantic checking (the latter when the command-line argument `--chk_decl` is specified) should be retained.
- Additionally, your program should construct an AST for each function in the input program. These ASTs should be printed out (see Sec. 3.2) if the command-line argument `--print_ast` is specified.

2. Driver code and program invocation

Your program should use the following driver file.

- [driver.c](#) -- a file containing the driver code for your parser. The driver scans the command-line arguments and sets the variable `print_ast_flag` to 1 if it is invoked with the argument `--print_ast`.

The command-line argument `--print_ast`, and thus the flag `print_ast_flag`, controls whether or not ASTs are printed out.

3. Abstract Syntax Trees

3.1. AST node types

In order to print your AST in a way that permits easy comparison with a reference implementation, you should use the `print_ast()` function provided (see below). In order to traverse your AST, this function needs to be able to recognize the type of each node. To this end, you should use the type `NodeType`, specified in the following header file, for your AST node types:

- [ast.h](#) -- header file for defining and interacting with ASTs.

This file mentions some node types (for arithmetic and Boolean operators) that are not in G2 but which will show up in subsequent assignments where we handle the entire C-- language. It's OK to just use the file as-is for now and ignore these additional node types.

3.2. Printing ASTs

Use the function `print_ast()`, defined in the file [ast-print.c](#) to print out your ASTs. This file mentions some node types (for arithmetic and Boolean operators) that are not in G2 but which show up in subsequent assignments where we handle the entire C-- language. It's OK to just use the file as-is and ignore these additional node types.

The AST for a function should be printed after its entire body has been processed and just before its local symbol table is discarded, e.g., something like:

```
/*
 * NOTE: code for semantic checking and AST construction are not shown
 * here. However, your implementation should include this code.
 */
void decl_or_func(char *name) {
    ...

    if (curr_tok == LPAREN) {      /* function definition */
        ...
        MATCH(LPAREN);
        opt_formals();
        MATCH(RPAREN);
        MATCH(LBRACE);
        opt_var_decls();
        opt_stmt_list();
        MATCH(RBRACE);
        /*
         * print the AST if requested
         */
        if (print_ast_flag) {
            print_ast( ... );
        }
        /*
         * reset the local symbol table
         */
        ...

    }
    else {      /* variable declaration */
        ...
    }
}
```

3.3. AST getter functions

In order to traverse your ASTs, the AST print function specified has to be able to access the children of each AST node. We could do this by specifying exactly how those children should be named, but this seems like unnecessary micromanagement. Instead, you are free to implement these data structures however you want, but must additionally implement the following getter functions (whose prototypes are defined in the file **ast.h**) for accessing their components:

- *NodeType ast_node_type(void *ptr)*
Given an arbitrary non-NULL AST pointer *ptr*, *ast_node_type(ptr)* returns the node type for the AST node that *ptr* points to.
- *char *func_def_name(void *ptr)*
*int func_def_nargs(void *ptr)*
*void *func_def_body(void *ptr)*
Given a pointer *ptr* to an AST for a function definition:
 - *func_def_name(ptr)* returns a pointer to the name of the function;
 - *func_def_nargs(ptr)* returns the number of formal parameters for the function;
 - *func_def_body(ptr)* returns a pointer to the AST of the body of the function.
- *char *func_def_argname(void *ptr, int n)*
Given: a pointer *ptr* to an AST for a function definition and an integer *n*, this function behaves as follows:
 - If $0 < n \leq$ no. of arguments for the function *ptr* refers to, *func_def_argname(ptr, n)* returns a pointer to the name (a string) of the n^{th} formal parameter for that function. The first formal parameter corresponds to $n = 1$.
 - Otherwise, the behavior of this function is undefined.
- *char *func_call_callee(void *ptr)*
*void *func_call_args(void *ptr)*
Given a pointer *ptr* to an AST node for a function call:
 - *func_call_callee(ptr)* returns a pointer to the name of the function being called;
 - *func_call_args(ptr)* returns a pointer to the AST for the list of arguments to the call.
- *void *stmt_list_head(void *ptr)*
*void *stmt_list_rest(void *ptr)*
Given a pointer *ptr* to an AST node for a statement list:
 - *stmt_list_head(ptr)* returns a pointer to the AST of the statement at the beginning of this list;
 - *stmt_list_rest(ptr)* returns a pointer to the AST of the rest of this list (i.e., to the next node in the list).
- *void *expr_list_head(void *ptr)*
*void *expr_list_rest(void *ptr)*
Given a pointer *ptr* to an AST node for an expression list:
 - *expr_list_head(ptr)* returns a pointer to the AST of the expression at the beginning of this list;
 - *expr_list_rest(ptr)* returns a pointer to the AST of the rest of this list (i.e., to the next node in the list).

- `char *expr_id_name(void *ptr)`
Given a pointer `ptr` to an AST node for an identifier, `expr_id_name(ptr)` returns a pointer to the name of the identifier (a string).
- `int expr_intconst_val(void *ptr)`
Given a pointer `ptr` to an AST node for an INTCONST, `expr_intconst_val(ptr)` returns the integer value of the constant.
- `void *expr_operand_1(void *ptr)`
`void *expr_operand_2(void *ptr)`
Given a pointer `ptr` to an AST node for an arithmetic or boolean expression.
`expr_operand_1(ptr)` returns a pointer to the AST of the first operand of the expression and `expr_operand_2(ptr)` returns a pointer to the AST of the second operand.
- `void *stmt_if_expr(void *ptr)`
`void *stmt_if_then(void *ptr)`
`void *stmt_if_else(void *ptr)`
Given a pointer `ptr` to an AST node for an IF statement:
 - `stmt_if_expr(ptr)` returns a pointer to the AST for the expression tested by the if statement;
 - `stmt_if_then(ptr)` returns a pointer to the AST for the then-part of the if statement, i.e., the statement to be executed if the condition is true;
 - `stmt_if_else(ptr)` returns a pointer to the AST for the else-part of the if statement, i.e., the statement to be executed if the condition is false.
- `char *stmt_assg_lhs(void *ptr)`
`void *stmt_assg_rhs(void *ptr)`
Given a pointer `ptr` to an AST node for an assignment statement:
 - `stmt_assg_lhs()` returns a pointer to the name of the identifier on the LHS of the assignment;
 - `stmt_assg_rhs()` returns a pointer to the AST of the expression on the RHS of the assignment.
- `void *stmt_while_expr(void *ptr)`
`void *stmt_while_body(void *ptr)`
Given a pointer `ptr` to an AST node for a while statement:
 - `stmt_while_expr()` returns a pointer to the AST of the expression tested by the while statement;
 - `stmt_while_body()` returns a pointer to the AST of the body of the while statement.
- `void *stmt_return_expr(void *ptr)`
Given a pointer `ptr` to an AST node for a return statement, `stmt_return_expr()` returns a pointer to the AST of the expression whose value is returned, NULL if there is no such expression.

3.4. Makefile

You should submit a Makefile that provides (at least) the following targets:

- **make clean:**

Deletes any object files (*.o) as well as the file 'compile'

- **make compile:**
Compiles all the files from scratch and creates an executable file named 'compile'.

Interface requirements

Your program should be written to interface appropriately with the driver code mentioned above. To this end, your code should provide the following:

```
int parse(); /* the parser function */
```

The driver code will set the value of an int variable `print_ast_flag` based on whether or not a command-line argument `--print_ast` is specified (see the code in the file **driver.c** linked above). The parser should call the function `print_ast()` to print out ASTs if and only if `print_ast_flag` has a non-zero value.

Your program should implement the AST getter functions specified in Section 3.3 above.

Files you need to submit

- Makefile that supports the functionality described above;
- the files provided for supporting the functionality requirements described above, i.e.:
 - `driver.c` -- the driver file provided to you for this assignment;
 - `ast.h` -- the header file containing AST node types and getter function prototypes;
 - `ast-print.c` -- the file containing the AST printing routine.
- all additional files implementing your scanner and parser.

Working with void *

You can convert a void * value to a pointer of a particular type by assigning it to a variable of the appropriate pointer type. For example:

```
NodeType ast_node_type(void *ptr) {  
    ASTnode *ast = ptr;  
    assert(ast != NULL);  
    return ast->ntype;  
}
```

Examples of ASTs for empty statement lists

Here are a few examples of programs with empty statement lists and what the autograder expects to see for each of them.

Input 1:

```
/*
 * A functions whose body is an empty statement list
 */

int f() {
}
```

Expected output 1:

```
func_def: f
  formals:
  body:
/* func_def: f */
```

Input 2:

```
/*
 * a function whose body consists of a sequence of empty stmt lists
 */

int f() {
  {}
  {}
  {}
}
```

Expected output 2:

```
func_def: f
  formals:
  body:
    {
    }
/* func_def: f */
```

Input 3:

```
/*
 * a function whose body consists of a single nested sequence of
 * empty stmt lists
 */

int f() {
    {
        {
            {}
        }
    }
}
```

Expected output 3:

```
func_def: f
  formals:
  body:
    {
      {
        {
        }
      }
    }
/* func_def: f */
```

Input 4:

```
/*
 * a function whose body consists of a sequence of nested empty stmt lists
 */

int f() {
    {
        {
            {}
        }
    }

    {
        {
            {}
        }
    }

    {
        {
            {}
        }
    }
}
```

Expected output 4:

```
func_def: f
  formals:
  body:
    {
      {
        {
        }
      }
      {
        {
        }
      }
      {
        {
        }
      }
    }
  /* func_def: f */
```